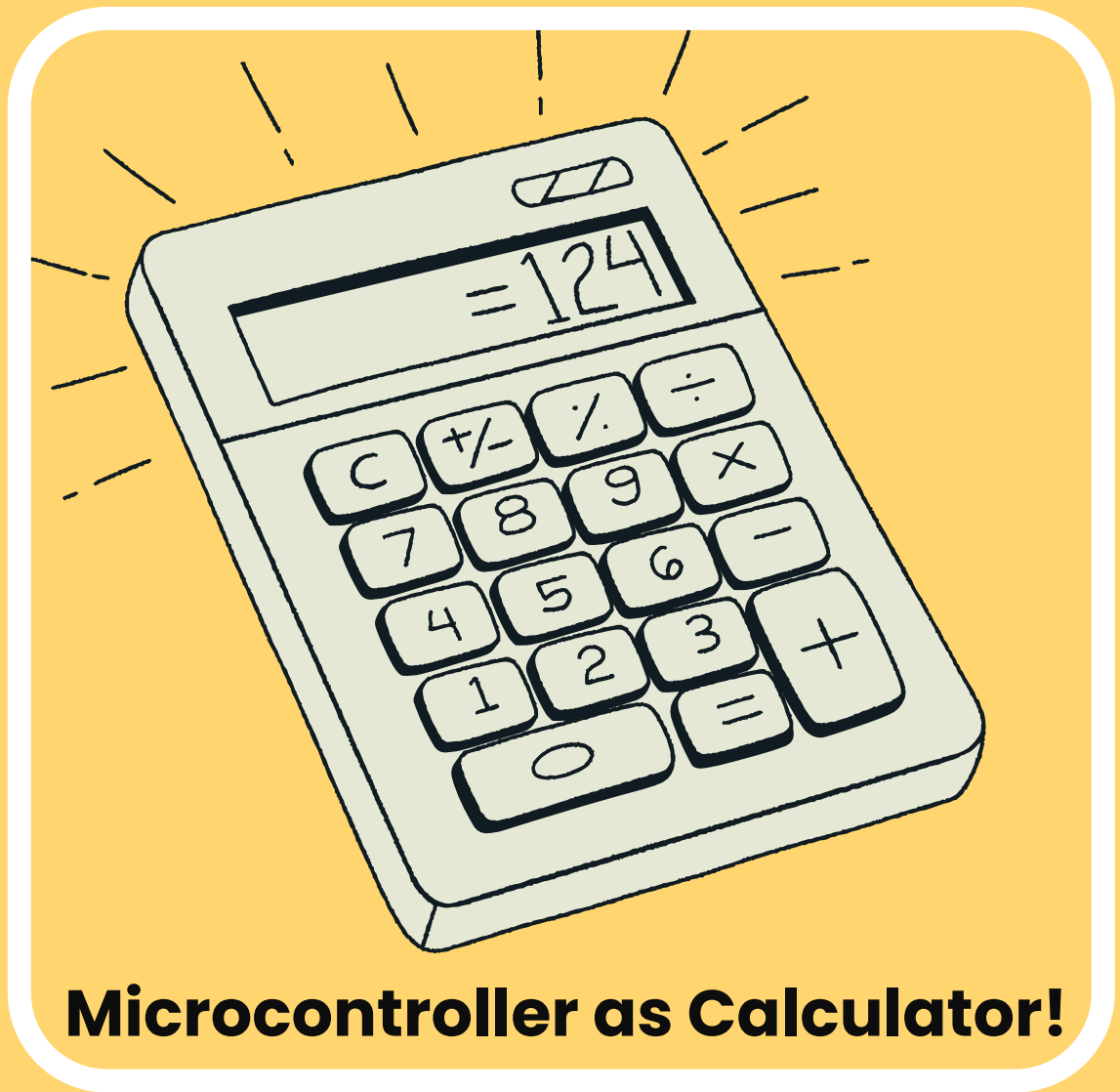
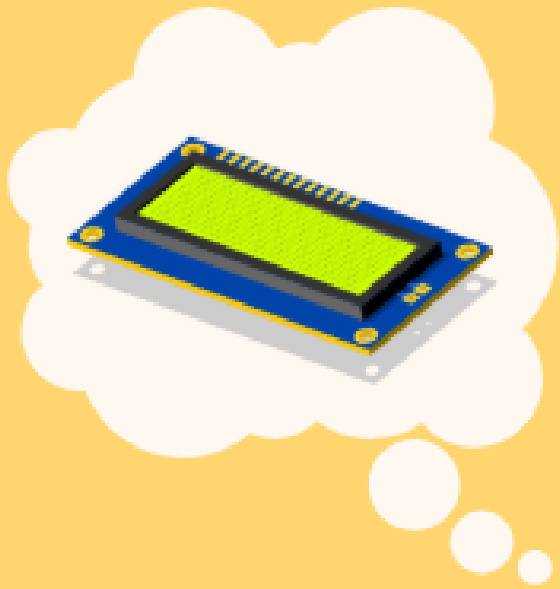




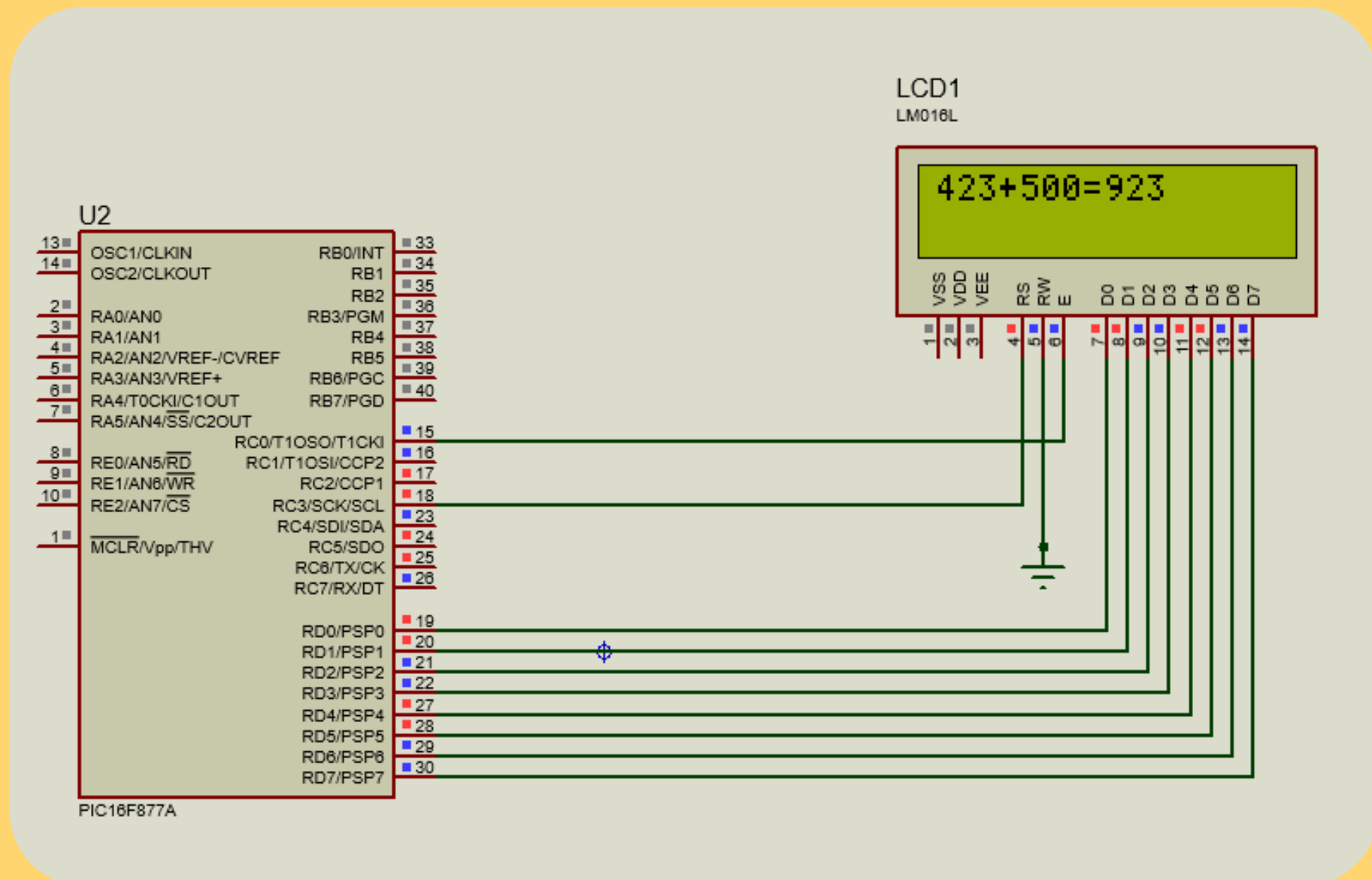
Microcontroller as Calculator!



[in/kapuluru-chaithanya](https://www.linkedin.com/in/kapuluru-chaithanya)

Goal:

- To calculate the addition of two numbers and display the result on the LCD.



- For simplicity, we are using the LCD's 8-bit mode and sending data through pins D0 to D7.
- We pull the RW pin low (0) by grounding it to enable the write operation.

Our initial task is to properly initialize the LCD.



```

void init(){
    TRISC = 0x00;//setting all pins as output
    TRISD = 0x00;//setting all pins as output
    delay_ms(100);
    lcd_command(0x30);//Function Set Command for 8 bit
    delay_ms(100);
    lcd_command(0x30);//Function Set Command for 8 bit
    delay_ms(100);
    lcd_command(8);//Function Set Command for 8 bit
    lcd_command(0x38);//Set lines(N) & font(F)
    lcd_command(0x0C);//Display on,cursor off ,blink off
    lcd_command(0x01);//Clear Display
}

```

- LCD initialization is done according to the sequence recommended in the datasheet.

```

void lcd_command(unsigned char i){
    RC3 = 0;    //RS pin set high (command mode)
    PORTD = i;
    RC0 = 1;    //Enable pin for clock set high
    delay_ms(100);
    RC0 = 0;    //Enable pin for clock set low
}

```

```

void lcd_data(unsigned char i ){
    RC3 = 1;    //RS pin set high (data mode)
    PORTD = i;
    RC0 = 1;    //Enable pin for clock set high
    delay_ms(100);
    RC0 = 0;    //Enable pin for clock set low
}

```

- The **lcd_command** function was used to send **instructions**, while **lcd_data** handled the **display characters**. Together, they worked as expected.

But there's a cleaner and more optimized way to write these functions!

Instead of updating RC3 explicitly, we can change its value using **bit manipulation** so the remaining bits stay unchanged.

By clearing and setting only those specific bits.



Set:

- To set the 3rd bit of PORTC, we use the bitwise OR (|) operation.

Let's assume PORTC contains the value 0x11.

PORTD	0	0	0	1	0	0	0	1
OR								
NUM	0	0	0	0	0	1	0	0
=								
Result	0	0	0	1	0	1	0	1

- Finally, assign the modified value back to PORTC.

$\text{PORTC} = \text{PORTC} | 0x08$ [i.e 0000 0100]

```
void lcd_data(unsigned char j){  
    PORTC |= 0x08;  
    PORTD = j;  
    PORTC |= 0x01;  
    __delay_ms(100);  
    PORTC &= ~(0x01);  
}
```



Clear:

- To clear the 3rd bit of PORTC, we use the bitwise AND (&) operation.

Let's assume PORTC contains the value 0xAC.

PORTD	1	0	1	0	1	1	0	0
-------	---	---	---	---	---	---	---	---

AND &

~ NUM	1	1	1	1	1	0	1	1
-------	---	---	---	---	---	---	---	---

=

Result	1	0	1	0	1	0	0	0
--------	---	---	---	---	---	---	---	---

- Finally, write the modified value back to PORTC.

$\text{PORTC} = \text{PORTC} \& \sim(0x08)$ [i.e 1111 1011]

```
void lcd_command(unsigned char i){  
    PORTC&= ~(0x08);  
    PORTD = i;  
    PORTC|= 0x01;  
    __delay_ms(100);  
    PORTC&= ~(0x01);  
}
```



```

void main(){
    unsigned int num1,num2;
    unsigned char plus,equal;
    plus = '+'; // Assign '+' symbol
    equal = '='; // Assign '=' symbol
    num1 = 423; // First number to display
    num2 = 500; // Second number to display

    init(); // Initialize LCD (configure, clear, and set display ON)

    lcd_command(0x80); // Set cursor to the first line, first position
    lcd_output(num1); // Display the first number (423)

    lcd_command(0x83); // Move cursor to column 4 (after first number)
    lcd_data(plus); // Display '+' symbol

    lcd_command(0x84); // Move cursor to next position
    lcd_output(num2); // Display the second number (500)

    lcd_command(0x87); // Move cursor to next position
    lcd_data(equal); // Display '=' symbol

    lcd_command(0x88); // Move cursor to next position
    lcd_output(num1 + num2); // Display the result of addition (423 + 500 = 923)
    while(1); // Infinite loop to keep displaying the result
}

```

- Executes the LCD initialization, positions the cursor, and displays the complete calculation.

```

void lcd_output(unsigned int num){
    unsigned char temp = 0, i = 0; // 'temp' stores each extracted digit
    // Extract digits from the number and store them in reverse order
    while(num != 0){
        temp = num % 10; // Get the last digit of the number
        array[i] = temp; // Store the digit in the array
        num = num / 10; // Remove the last digit from the number
        i++; // Move to the next index in the array
    }

    array[i] = '\0'; // Null-terminate the array (end marker)

    // Send digits to LCD in the correct order
    while(i > 0){
        i--; // Decrement index to access digits in reverse (original) order
        lcd_data(0x30 + array[i]); // Convert digit to ASCII ('0' = 0x30)
    }
}

```

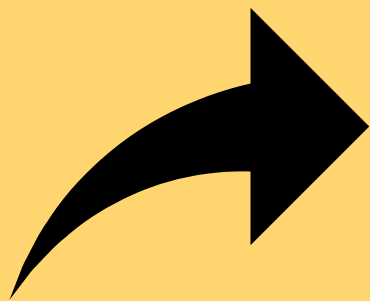
- Converts the given number into individual digits and stores them in reverse order.
- Then sends each digit as an ASCII character to the LCD for proper display.

There's more than one way to achieve the same result!

Now, let's bring all these functions together to achieve our goal.



Learn.
Grow.
Connect.



↕ Repost



[in/kapuluru-chaithanya](https://www.linkedin.com/in/kapuluru-chaithanya)

